Sanskrit Document RAG System: Technical Report

Nov 7, 2026 · Nishikant Dalal

1. Introduction

This report describes a Retrieval-Augmented Generation (RAG) system that answers questions about five short Sanskrit stories using only a CPU. Prepared by Nishikant Dalal. The system accepts questions in Devanagari or Roman letters, retrieves the most relevant passages, picks the best-matching sentence, and asks a small local language model to produce an answer.

The main results, measured on 19 test questions on an Intel Core i3-1125G4 laptop with 8 GB RAM and no GPU:

- Hybrid retrieval (embeddings plus keyword search) found a correct passage in the top 3 for all 19 questions (Hit@3 = 1.0, MRR = 0.85).
- Selecting a single best sentence before generation doubled the evidence match from 32% (6 of 19) to 63% (12 of 19) and cut answer time from 16.0 s to 7.1 s.
- Peak memory was about 1.2 GB, and the whole pipeline runs offline.
- The main weakness is the 1.5B-parameter model's understanding of Sanskrit: its English renderings are often wrong even when the retrieved sentence is right.

2. System architecture

The system has two phases. Indexing runs once; querying runs for every question.

Indexing (run once)

1. Read `data/raw/*.txt`.
2. Clean the text (Devanagari only) and split it at danda marks.
3. Group the sentences into overlapping 3-sentence chunks.
4. Embed each chunk with E5 and store the vectors in a FAISS index.
5. Save the chunk list to `index/chunks.json`; BM25 is built from it at start-up.

Query (every question)

1. Transliterate a Roman-letter question to Devanagari.
2. Retrieve the top 3 chunks with hybrid search: dense (FAISS) plus BM25 n-grams, merged by RRF.
3. Pick the single best-matching sentence (BM25 over the sentences).

4. Prompt the local LLM (llama.cpp, 4-bit, CPU) with that sentence.
5. Output the best sentence, the LLM answer, the retrieved passages and the timings.

The code is split into modules with one job each:

Module	Responsibility
`preprocess.py`	Cleaning, sentence splitting, chunking, transliteration
`indexer.py`	Builds the FAISS vector index
`retriever.py`	Dense, BM25 and hybrid search
`extractive.py`	Selects the best sentence from the retrieved chunks
`generator.py`	Wraps the quantised LLM (llama.cpp)
`rag_pipeline.py`	The only module that connects retriever and generator, and times each stage
`main.py` , `app_gradio.py`	Command-line interface and optional web page
`evaluate.py` , `auto_score.py`	Metrics

The retriever and generator never call each other. Only `rag_pipeline.py` does, so either can be replaced (a different vector store, or a different LLM) without touching the other. All settings live in `config.yaml` . The generator is forced onto the CPU (`n_gpu_layers=0` , and the embedding model runs with `device="cpu"`).

3. Documents

The corpus is one Word file supplied with the assignment, `Rag-docs.docx` , containing five short Sanskrit stories in Devanagari. A script (`convert_docx.py`) splits it into one UTF-8 text file per story. After cleaning, the corpus has 120 sentences, about 1,000 words and 59 chunks.

Story (file)	Content	Sentences	Chunks	Words
`01_murkhabhritya`	The foolish servant Shankhanada misreads his master's instructions	22	11	195
`02_kalidasa_lakshmudra`	Kalidasa helps a poet win King Bhoj's reward	20	10	177

Story (file)	Content	Sentences	Chunks	Words
`03_vriddha_chaturya`	An old woman defeats the bell-demon Ghantakarna	20	10	172
`04_devabhakta`	A devotee who only prays and makes no effort	45	22	310
`05_shitam_bahu_badhati`	Kalidasa, disguised as a palanquin bearer, and a foreign scholar	13	6	142
Total		**120**	**59**	**996**

The raw text is not clean, which shaped the preprocessing:

- Two stories contain English translations, in brackets or inline, mixed with the Sanskrit.
- Verses are marked with `//` instead of danda marks.
- There are spelling variants and typos, such as `सन्चिकायाम्` and a stray `स्`. These were deliberately left uncorrected, to test the system on realistic text.
- Two titles carry an author credit and an e-mail address, which are removed during conversion.

4. Preprocessing pipeline

The pipeline in `preprocess.py` runs in four steps.

1. **Cleaning.** Text is normalised to Unicode NFC and zero-width characters are removed. Bracketed English is deleted, then any remaining run of Latin text, so only Devanagari stays. Quote marks and brackets are removed, and the `//` verse marker becomes a danda (`।`).
2. **Sentence splitting.** Text is split after each danda (`।`) or double danda (`॥`). English full stops cannot be used, because Sanskrit text does not use them. Fragments with fewer than three Devanagari characters are dropped. The result is 120 sentences.
3. **Chunking.** Sentences are grouped into windows of 3 with an overlap of 1 sentence, giving 59 chunks. Overlap matters because Sanskrit sentences often drop their subject (for example `सः आपणम् गच्छति`, "he goes to the shop"), which is named in the sentence before. Each chunk keeps its story name and an ID so answers can cite their source.
4. **Query transliteration.** Queries in Roman letters are converted to Devanagari with the `indic-transliteration` library. ITRANS is the default scheme (`ghaNTAkarNaH kaH` becomes `घण्टाकर्णः कः`). Input containing diacritics such as `ā`, `ī`, `ś` is detected as IAST. Devanagari input is used unchanged.

Example of the cleaning step. The raw text contains the Sanskrit sentence followed by an English translation in brackets ("King Bhoj once proclaimed ..."). After cleaning only the Sanskrit remains:

```
घोषितं कदाचित् भोजराज्ञा, यदि कोऽपि कविः मम दरबारे नूतनं काव्यं पठति तर्हि ददामि तस्मै लक्षरुप्यकाणि इति ।
```

No Sanskrit stop-word list or sandhi splitter was used. Instead, retrieval relies on character n-grams to tolerate inflected and joined word forms (Section 5).

5. Retrieval

Three retrieval modes are implemented and compared.

- **Dense.** Each chunk is embedded with `intfloat/multilingual-e5-small` (about 470 MB, run on CPU) and stored in a FAISS inner-product index. Vectors are normalised, so the score is cosine similarity. Queries and passages use the E5 prefixes `query:` and `passage:`.
- **BM25 on character n-grams.** Each word and its character 3-grams are indexed (`rank_bm25`). Sanskrit words inflect and join, so exact-word matching misses forms such as `शर्कराम्` versus `शर्करा`. The n-grams make matching tolerant of these changes.
- **Hybrid.** The two ranked lists (10 candidates each) are merged with Reciprocal Rank Fusion, where each method has a weight:

A chunk's fused score is the sum, over the two methods, of the method's weight divided by 60 plus the chunk's rank in that method's list: score = w_dense / (60 + rank_dense) + w_bm25 / (60 + rank_bm25). A chunk missing from one list gets no contribution from that method.

The top 3 chunks are returned.

Evaluation

The test set has 19 questions: 12 that follow the wording of the stories, plus 7 harder ones that reword the story or are typed in Roman letters. Each question has a gold phrase, and a retrieval is a hit if a returned chunk contains it. Hit@1 and Hit@3 are the share of questions with a hit in the top 1 or top 3 chunks. MRR is the mean of 1 divided by the rank of the first hit.

Mode	Hit@1	Hit@3	MRR	Avg. search time
Dense (E5 + FAISS)	0.53	0.84	0.66	31.0 ms
BM25 (character n-grams)	0.74	0.89	0.79	0.7 ms

Mode	Hit@1	Hit@3	MRR	Avg. search time
Hybrid (RRF, dense weight 0.3)	0.74	1.00	0.85	25.6 ms

Hybrid is the only mode that placed a correct chunk in the top 3 for every question, so it is the default.

Tuning the hybrid weights

On the first 12 questions, hybrid with equal weights scored Hit@1 0.75, Hit@3 0.92 and MRR 0.83. That was worse than BM25 alone (0.92, 1.00, 0.94), and dense search alone was worst (0.58, 0.75, 0.65). Reducing the dense weight to 0.3 raised hybrid to 0.92, 1.00 and 0.94, equal to BM25 on that set. The extra questions then separated the methods in favour of hybrid.

The weak dense result suggests that this small multilingual embedding model represents Sanskrit poorly, probably because Sanskrit is thinly covered in its training data. This was not tested against other embedding models. With only 19 questions, a single question moves Hit@1 by 5 points, so small differences should not be over-read.

6. Generation

Model. The generator is Qwen2.5-1.5B-Instruct in a 4-bit quantised GGUF file (Q4_K_M, 1.12 GB), run through `llama-cpp-python` (llama.cpp). Quantisation shrinks the weights from 16 bits to about 4 bits each, which makes CPU inference practical on 8 GB of RAM. GPU offload is disabled (`n_gpu_layers=0`). The context window is 2048 tokens and the temperature is 0.2 for stable, grounded output.

Prompt. The system prompt tells the model to use only the passage it is given, to answer in one or two short English sentences, to add an `Evidence:` line that copies the supporting Sanskrit sentence, and to reply "The answer is not in the documents" when the passage lacks the answer. One worked example is included in the prompt. The first version asked for a Sanskrit answer plus an English translation. In a test, the model ignored that format and cited a passage number that did not exist, so the prompt was changed to something a small model can follow: an English answer and a quoted Sanskrit sentence.

Best-sentence selection. In the first design the model received three retrieved chunks (about nine sentences) and often misread them. An extra step was added in `extractive.py`: it splits the retrieved chunks into sentences, scores each with BM25 against the query, and keeps the highest-scoring sentence. The model then sees only that sentence, and the final output shows the sentence first, followed by the model's line. The sentence is selected by code, so it is an exact quote from the corpus.

Design	Evidence match (19 questions)	Avg. generation time	Speed
LLM reads 3 retrieved chunks	6 of 19 (32%)	16.0 s	4.8 tokens/s
Best sentence selected, then LLM	12 of 19 (63%)	7.1 s	9.4 tokens/s

The evidence match is an automatic score: an answer counts as a match when it contains the gold phrase for that question. It is not human grading. It can undercount correct answers that quote a different but equally valid sentence, and it can overcount answers that contain the phrase but misread it.

7. Performance

The system answers a question in about 7 seconds on a low-end laptop CPU and uses about 1.2 GB of RAM. All figures below come from the 19-question evaluation (`evaluate.py`).

Measure	Result
Hardware	Intel Core i3-1125G4 (4 cores, 8 threads), 8 GB RAM, Windows 11, no GPU, Python 3.12.10
Retrieval time per question	about 0.1 s (transliteration, hybrid search, sentence selection)
Generation time per question	7.1 s on average
Generation speed	9.4 tokens per second
Peak process memory	about 1.2 GB
Model files on disk	LLM 1.12 GB, embedding model about 470 MB
Retrieval quality	Hit@3 = 1.00, MRR = 0.85 (hybrid)
Evidence match	12 of 19 (63%)

No human grading was done: the only answer-quality measure is the automatic evidence match described in Section 6.

Failure analysis

Seven of the 19 questions did not match. Because retrieval found a correct chunk in the top 3 every time, the failures come from sentence selection and from the language model, not from search. They fall into three groups, and some failures show more than one.

Question	Sentence selected (meaning)	What went wrong
Who shook the bell?	"Out of curiosity, [they] took the bell in hand and shook it."	The subject, the monkeys, is named in the previous sentence. The model answered "The sun rises in the evening."
How did the puppy die?	"And that puppy dies."	The cause, a blocked breath, is in the sentence before. The model answered "The swan is dead", confusing `श्वान` (dog) with a swan.
What did King Bhoj announce?	The opening line of a poem given to a new poet	The wrong sentence won on word overlap. The model answered "praised for being a truthful religious teacher".
How did the old woman kill the demon?	"Hearing that, an old woman went to the forest."	The key step, taking the bell while the monkeys ate fruit, comes several sentences later. The model answered "has lost her finger".

In summary: (1) the answer sits in a neighbouring sentence, because single-sentence selection cuts context; (2) keyword overlap sometimes picks the wrong sentence; (3) even when the right sentence is shown, the 1.5B model's English rendering is often wrong. This is why the Sanskrit sentence is printed first, as the part of the output the user can trust.

8. Limitations and future work

Limitations

- **Small corpus and test set.** Five stories and 19 questions. The questions were written by the author from the same text, so scores are likely optimistic, and one question moves a metric by 5 points.

- **Tuning on the test set.** The dense weight in the hybrid search was chosen using the first 12 questions, which are part of the final 19, so the hybrid result may be slightly flattering.
- **Weak generator.** The 1.5B model writes unreliable English renderings of Sanskrit sentences. A larger model was not tested.
- **Context loss.** Single-sentence selection sometimes drops the sentence that names the subject or gives the cause.
- **No Sanskrit-specific language tools.** No sandhi splitter, morphological analyser or stop-word list was used.
- **Automatic scoring only.** The evidence match checks for a phrase, not for correctness of meaning, and no human grading was done.
- **Roman input.** Typed queries must follow the ITRANS or IAST conventions, otherwise the transliteration is wrong.

Future work

- Test a 3B or larger quantised model, or one with better Sanskrit support, and compare it in the same table.
- Return the selected sentence together with its neighbours, to recover missing subjects and causes.
- Add a cross-encoder re-ranker for sentence selection.
- Try embedding models trained on Indic languages, and add a sandhi splitter before indexing.
- Build a larger, independently written test set and add human scoring of answers.

9. Conclusion

A complete Sanskrit RAG system runs on a low-end laptop CPU, using about 1.2 GB of RAM and about 7 seconds per answer. Hybrid retrieval over character n-grams and embeddings found a correct passage in the top 3 for all 19 test questions, and selecting the best sentence before generation doubled the evidence match from 32% to 63%. The remaining weakness is the small language model's grasp of Sanskrit, which is why the system shows the exact Sanskrit sentence it found alongside the model's answer. The retriever and generator are separate modules, so a stronger model can be swapped in through `config.yaml` without redesigning the system.

Code and reproduction

The repository contains the code, the five story files, the evaluation questions and a README with setup and run steps: `python code/main.py index`, `python code/download_model.py`, then `python code/main.py ask "<question>"`. Evaluation is run with `python code/evaluate.py --with-llm` followed by `python code/auto_score.py`.